

Bases de Datos

Clase 16: Pipelining y Planificación de Consultas

Agenda

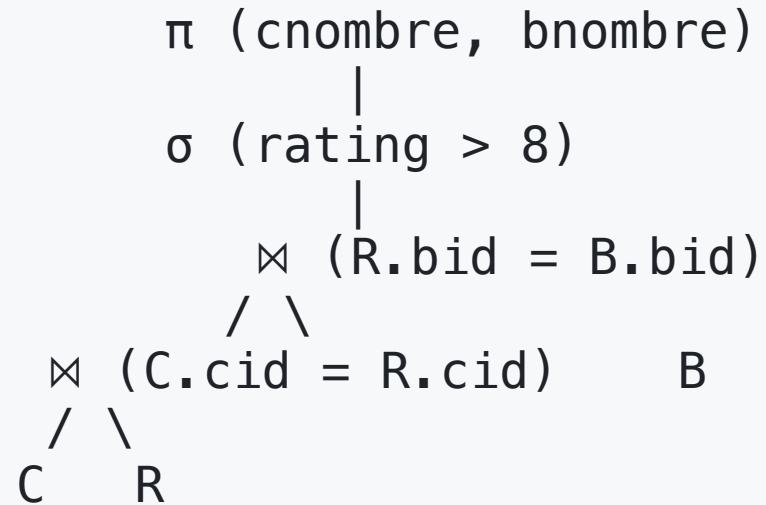
1. Pipelining

2. Del Plan Lógico al Plan Físico

Pipelining

Motivación

```
SELECT C.cnombre, B.bnombre  
FROM Capitanes C, Reservas R, Botes B  
WHERE C.cid = R.cid AND R.bid = B.bid  
AND C.rating > 8
```



¿Cómo se **ejecuta** este árbol?

Dos estrategias de ejecución

Materialización:

- Cada operador se ejecuta hasta el final, escribe su salida en disco, el siguiente la lee.
- Pagamos I/O por cada resultado intermedio.

Pipelining:

- Los operadores corren simultáneamente; cada uno emite tuplas tan pronto las produce.
- Sin escribir resultados intermedios completos.

El modelo de iteradores

Cada operador físico implementa **tres funciones**:

Función	Qué hace
<code>open ()</code>	Inicializa el operador (reserva recursos, abre hijos)
<code>next ()</code>	Retorna la siguiente tupla del resultado (o null si terminó)
<code>close ()</code>	Libera recursos

El operador padre llama `next ( )` al hijo para obtener tuplas **una a una**.

Pipelining en acción

```
π.next() ← "dame una tupla"
      |
σ.next() ← "dame una tupla que cumpla rating > 8"
      |
⋈.next() ← "dame la siguiente tupla del join"
    /  \
⋈.next() B.next()
  /  \
C.next() R.next()
```

Las tuplas fluyen **de abajo hacia arriba** bajo demanda — no hay escrituras intermedias.

Selección como iterador (repaso)

```
open()    → R.open()
next()    → t := R.next()
           while t ≠ null do
             if t satisface condición then return t
             t := R.next()
           return null
close()   → R.close()
```

Cada llamada a `next()` retorna **una tupla** → el operador padre puede usarla inmediatamente.

¿Todos los operadores pueden hacer pipelining?

Sí (pipeline-friendly):

- σ y π : emiten al vuelo.
- Nested Loop / BNLJ: producen tuplas a medida que encuentran matches.

No completamente (blocking / materializan):

- **Sort (External Merge Sort)**: debe ver TODAS las tuplas antes de emitir la primera.
- **Hash Join — fase de build**: debe particionar todo antes de empezar el probe.
- **Agregación con GROUP BY**: debe terminar todos los grupos antes de emitir.

Los operadores blocking son los "cuellos de botella" del pipelining.

Impacto del pipelining en costos

External Merge Sort cuesta $\sim 4N$ I/O:

- Fase 0: leer N + escribir $N = 2N$
- Fase 1: leer N + escribir $N = 2N$

Si el **operador padre** consume las tuplas directamente del merge final $\rightarrow$ no escribimos la última fase:

- Costo efectivo $\approx 3N$ (ahorramos N escrituras).

Del Plan Lógico al Plan Físico

Ciclo completo de una consulta

SQL → Parser → Plan Lógico → Optimizador → Plan Físico → Ejecución (pipelining)

1. **Parser:** convierte SQL a álgebra relacional (plan lógico).
2. **Optimizador:** elige el plan físico más eficiente.
3. **Ejecución:** ejecuta el plan con iteradores y pipelining.

El **optimizador** decide: qué algoritmo usar para cada operador, en qué orden hacer los joins, dónde empujar las selecciones.

Reescritura: empujar selecciones hacia abajo

Antes:

```
σ (rating > 8)
  |
⋈ (C.cid = R.cid)
 /  \
C    R
```

Después:

```
⋈ (C.cid = R.cid)
 /  \
σ(rating>8)  R
 |
C
```

Filtrar **antes** del join reduce el tamaño de la entrada que el join procesa. Regla general: las selecciones bajan lo más posible.

Reescritura: el orden de los joins importa

```
SELECT ...  
FROM R, S, T  
WHERE R.a = S.a AND S.b = T.b
```

Tres órdenes posibles:

1. $(R \bowtie S) \bowtie T$
2. $(R \bowtie T) \bowtie S$ — sin condición directa entre R y T $\rightarrow$ producto cartesiano parcial.
3. $(S \bowtie T) \bowtie R$

Si $R = 1.000$ págs, $S = 10$ págs, $T = 500$ págs: empezar por $(R \bowtie S)$ da intermedio chico; empezar por $(R \bowtie T)$ puede ser catastrófico.

El espacio de planes explota rápido

Para n relaciones:

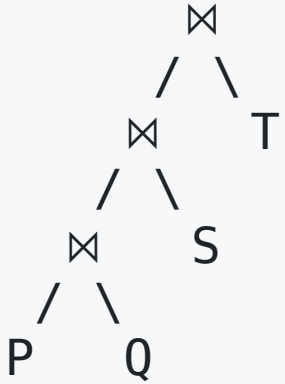
- $n!$ permutaciones del orden de joins.
- Por cada permutación, varias formas de **parentizar** el árbol (números de Catalan).
- Total $\approx n! \cdot C(n-1)$ árboles distintos.

n	Planes posibles
3	12
5	1.680
8	~17 millones
10	~17 mil millones

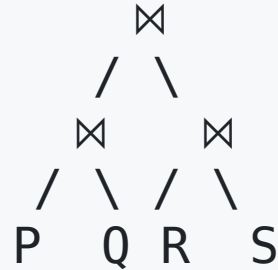
Imposible enumerarlos todos. El optimizador debe **podar el espacio**.

Árboles left-deep: heurística clave

Left-deep:



Bushy:



En un árbol **left-deep**, el lado derecho de cada join es siempre una relación base.

- Reduce el espacio de $n! \cdot C(n-1)$ a solo $n!$ permutaciones.
- Encaja perfecto con pipelining: el lado derecho se lee una vez, el lado izquierdo se construye incrementalmente.
- Para hash join: el lado derecho puede ser el build, el izquierdo el probe.

Por qué los DBMSs (casi) siempre eligen left-deep

System R (IBM, 1979) introdujo la heurística, y sigue siendo el default en PostgreSQL, Oracle, SQL Server.

Ventajas:

- Espacio de búsqueda manejable.
- Algoritmo de programación dinámica eficiente (siguiente slide).
- Pipelining natural: nunca hay que esperar a que termine un sub-join "a la derecha".

Cuándo se rompe: queries con joins muy selectivos en pares específicos (estrella o copo de nieve en data warehousing). Algunos optimizadores modernos (PostgreSQL `geqo`, ClickHouse) consideran árboles bushy en esos casos.

Programación dinámica sobre left-deep

Construir el mejor plan **bottom-up**:

Mejor plan para k relaciones = mejor join entre el mejor plan para $k-1$ y una relación más.

Costo: $O(n \cdot 2^n)$ en lugar de $O(n!)$.

n	$n!$	$n \cdot 2^n$
5	120	160
10	3.6 M	10.240
15	1.3 T	~500 K

Equivalencias del álgebra relacional

El optimizador usa estas reglas para reescribir el plan:

Regla	Descripción
Conmutatividad del join	$R \bowtie S = S \bowtie R$
Asociatividad del join	$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$
Empujar selección	$\sigma_c(R \bowtie S) = \sigma_c(R) \bowtie S$ (si c solo refiere a R)
Empujar proyección	$\pi_A(R \bowtie S)$ — se puede proyectar atributos no necesarios antes del join
Descomponer selección	$\sigma_{\{c1 \text{ AND } c2\}}(R) = \sigma_{\{c1\}}(\sigma_{\{c2\}}(R))$

Estadísticas del catálogo

Para estimar el costo de un plan, el optimizador necesita **estadísticas**:

Estadística	Para qué sirve
Páginas(R)	Costo de full scan
Tuplas(R)	Costo de NLJ
Valores distintos de R.a	Estimar selectividad de filtros y joins
Histogramas	Distribución de valores (no uniforme)
Profundidad del índice	Costo de búsqueda con índice

PostgreSQL las mantiene en `pg_statistic` / `pg_stats` , actualizadas con `ANALYZE` .

Estimación de cardinalidad

¿Cuántas tuplas produce un join?

Estimación simple (asumiendo distribución uniforme):

$$|R \bowtie S| \approx \frac{|R| \times |S|}{\max(V(R, a), V(S, a))}$$

Donde $V(R, a)$ = número de valores distintos de a en R .

Ejemplo:

- R tiene 10.000 tuplas, $V(R, a) = 500$
- S tiene 5.000 tuplas, $V(S, a) = 1.000$
- Resultado estimado $\approx 10.000 \times 5.000 / 1.000 = 50.000$ tuplas

Estimaciones imprecisas son una causa común de planes malos — EXPLAIN
ANALYZE las contrasta con la realidad.

El optimizador compara planes

Para la consulta:

```
SELECT C.cnombre FROM Capitanes C, Reservas R
WHERE C.cid = R.cid AND C.rating > 8
```

Plan	Descripción	Costo estimado
A	Full scan C $\rightarrow \sigma(\text{rating}>8) \rightarrow \text{NLJ con R}$	Alto
B	Índice en C.rating $\rightarrow \sigma(\text{rating}>8) \rightarrow \text{BNLJ con R}$	Medio
C	$\sigma(\text{rating}>8)$ en C $\rightarrow \text{HJ con R}$	Bajo (push-down aplicado)
D	Full scan C $\rightarrow \text{HJ con R} \rightarrow \sigma(\text{rating}>8)$	Medio-alto

Enumera planes candidatos (sobre el espacio left-deep), estima costos con las estadísticas, elige el más barato.

Resumen

El viaje completo de las consultas

```
SQL → Parser → Álgebra Relacional (plan lógico)
                |
                Reescritura
                (empujar  $\sigma$ , reordenar  $\bowtie$ )
                |
                Plan lógico optimizado
                |
                Optimizador
                (DP sobre left-deep, estadísticas)
                |
                Plan físico
                (BNLJ, HJ, Seq Scan, Index Scan...)
                |
                Ejecución
                (iteradores + pipelining)
```


¿Podemos ver el plan que eligió PostgreSQL?

Sí → `EXPLAIN` y `EXPLAIN ANALYZE`

```
EXPLAIN SELECT C.cnombre  
FROM Capitanes C, Reservas R  
WHERE C.cid = R.cid AND C.rating > 8;
```

Muestra el plan **sin ejecutar** la consulta.

```
EXPLAIN ANALYZE SELECT ...
```

Ejecuta la consulta y muestra costos estimados vs. **tiempos reales**.

→ La próxima clase es un **laboratorio práctico** con EXPLAIN.

Preview: qué veremos en EXPLAIN

```
Hash Join (cost=... rows=... width=...)
  Hash Cond: (r.cid = c.cid)
    → Seq Scan on reservas r (cost=... rows=...)
    → Hash (cost=... rows=...)
      → Seq Scan on capitanes c (cost=... rows=...)
        Filter: (rating > 8)
```

- Qué algoritmo eligió el optimizador (Hash Join, Nested Loop, Merge Join...).
- Si usa Seq Scan o Index Scan.
- Estimaciones de filas vs. realidad.
- Si agregar un índice **cambia el plan**.

Resumen de todo lo que vimos en algoritmos

Semana	Tema	Concepto clave
7	Almacenamiento	Costo I/O = páginas leídas/escritas
7	Índices	Hash (igualdad), B+ Tree (igualdad + rango)
8	Joins I	NLJ, BNLJ, INLJ
8	Joins II	SMJ, HJ, Grace HJ
9	Sorting + Selección	External Merge Sort, selección con/sin índice
10	Hoy	Pipelining, planificación, optimizador

Próxima clase: laboratorio con EXPLAIN ANALYZE → conectar la teoría con la práctica.

Ejercicio para cerrar

Consulta:

```
SELECT E.nombre, D.departamento  
FROM Empleados E JOIN Departamentos D ON E.did = D.did  
WHERE E.salario > 80000 AND D.ciudad = 'Santiago'
```

Datos: Empleados tiene 100.000 páginas. Departamentos tiene 50 páginas. Buffer de 200 páginas. B+ Tree en E.did. No hay índice en salario ni ciudad.

1. Dibuje el plan lógico (árbol de álgebra relacional).
2. Aplique reescritura: ¿dónde empujar las selecciones?
3. ¿Qué algoritmo de join elegiría? Justifique con costos.
4. ¿Qué índice agregaría para mejorar el plan?